

# **Advanced Dynare**

## Chapter I

### *The software Dynare*

Johannes Pfeifer

University of the Bundeswehr Munich

October 2025, Bank of Korea



# Outline

1. Intro

2. Dynare Overview

3. Modeling language

4. Invocation and results

# Licensing and Thank you

- The slides have benefited from the work and materials generously shared by friends and colleagues
  - Stéphane Adjemian
  - Houtan Bastani
  - Michel Juillard
  - Willi Mutschler
  - Philipp Pfeiffer
  - Marco Ratto
  - Normann Rion
  - Sébastien Villemot
- The licensing varies between completely free, MIT, CC-BY-SA 4.0, and unspecified  
→ impossible to sort out in time
- Where applicable, I state the license  
→ this chapter benefitted from Philipp Pfeiffer

# Licensing and Thank you

- Slides always contain errors
- If you spot one, please let me know
- I will correct them in red

# Outline

1. Intro
2. Dynare Overview
3. Modeling language
4. Invocation and results

# What is Dynare?

- A free software platform for handling a wide class of economic models, in particular, DSGE models
- Provides modeling language to represent models and computing tasks, with clear separation between
  1. particular model declared by the user
    - binary preprocessor to translate modeling language into model representation useful for computations
  2. computing functions that handle an entire family of DSGE models
    - collection of MATLAB/Octave routines for computations/task
- Dynare is very user-friendly and has low entry costs
  - history is described in the aptly titled **CherrierEtAl2023: CherrierEtAl2023**
- Users provide .mod file containing parameters, model equations linking the variables, and computing tasks to be performed

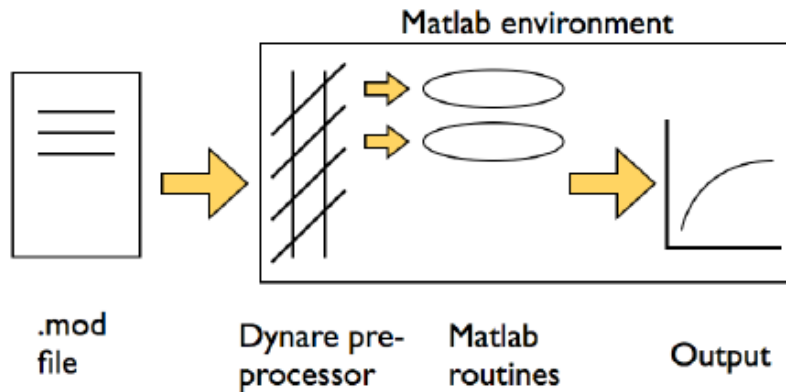
# Dynare resources

- Download at <https://www.dynare.org/>, latest version: Dynare 6.4  
→ Dynare 7 hopefully in Q1 of 2026
- Dynare manual: Adjemian et al. (2024)
- Forum at <https://forum.dynare.org/>
- Git repository with code, Wiki, buglist at <https://git.dynare.org/Dynare/dynare/>
- Pfeifer (2013a): *A Guide to Specifying Observation Equations for the Estimation of DSGE Models*
- Pfeifer (2013b): *An Introduction to Graphs in Dynare*
- Collection of example files: [https://github.com/johannespfeifer/dsge\\_mod](https://github.com/johannespfeifer/dsge_mod)

## Tools available in Dynare

1. computes the solution of perfect foresight models (arbitrary accuracy)
2. computes first- and higher-order approximation to solution of stochastic models
3. estimates (maximum likelihood, Bayesian approach, method of moments) parameters of DSGE models, for linear and non-linear models
4. handles occasionally binding constraints
5. checks for identification of estimated parameters
6. computes optimal policy (OSR, Ramsey, discretion)
7. performs global sensitivity analysis of a model
8. provides macro language and reporting facility
9. forecasting

## How Dynare works



# Outline

1. Intro
2. Dynare Overview
3. Modeling language
4. Invocation and results

# A Dynare file

Our model:

- Endogenous variables:  $\pi_t, y_t, i_t, \varepsilon_t^a, \varepsilon_t^c, \varepsilon_t^i$
- Some parameters:  $\beta, \kappa, \sigma, \phi_\pi, \phi_y, \rho^a, \rho^c, \rho^i$
- Exogenous variables (shocks):  $u_t^a, u_t^c, u_t^i$
- A three equation New Keynesian model:

$$\pi_t = \beta \mathbb{E}_t [\pi_{t+1}] + \kappa y_t + \varepsilon_t^a \quad (1)$$

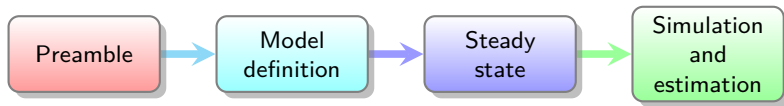
$$y_t = \mathbb{E}_t [y_{t+1}] - \frac{1}{\sigma} (i_t - \mathbb{E}_t [\pi_{t+1}]) + \varepsilon_t^c \quad (2)$$

$$i_t = \phi_\pi \pi_t + \phi_y y_t + \varepsilon_t^i \quad (3)$$

and AR-processes for exogenous processes

$$\varepsilon_t^j = \rho_j \varepsilon_{t-1}^j + u_t^j \quad (4)$$

# Structure of a Dynare file



1. Preamble: Declaration of endogenous and exogenous variables and parameters. Set value for calibrated parameters
2. Model definition: State the dynamic model equations
3. Steady state (long-run equilibrium)
4. Additional computations: model checks, simulation, estimation, ...
  - All command lines are terminated with a semicolon
  - Blocks are terminated with an `end`;
  - Comments either using `%` (Matlab style) or `//`, `/* */` (C-style)  
→ Matlab comments allow simpler debugging

# How to interact with Dynare: 1) Preamble

- Endogenous variables:  $\pi_t, y_t, i_t, \varepsilon_t^a, \varepsilon_t^c, \varepsilon_t^i$
- Exogenous variables (shocks):  $u_t^a, u_t^c, u_t^i$
- Parameters:  $\beta, \theta, \eta, \sigma, \phi_\pi, \phi_y, \rho^a, \rho^c, \rho^i$

Dynare code

```
var // endo vars
    pie y inom eps_a eps_c eps_i;

varexo
    u_a u_c u_i;

parameters
    betta theta eta sig phi_pi
    phi_y rho_a rho_c rho_i;
```

## L<sup>A</sup>T<sub>E</sub>X definitions in Dynare

- Note: For all three types, you can specify L<sup>A</sup>T<sub>E</sub>X names with \$\$ and long names for Dynare output with (long\_name='name here')

```
var // endo vars
pie ${\pi}$ (long_name='Quarterly inflation rate')
y   ${y}$   (long_name='Output gap')
;

varexo
u_a ${u_a}$ (long_name='Cost-push shock')
;

parameters
beta   ${\beta}$   (long_name='Discount factor')
theta  ${\theta}$   (long_name='Calvo parameter')
;
```

## How to interact with Dynare: 1) Preamble (cont'd)

- Set  $\beta$  and  $\theta$  to conventional values informed by real interest rates and price adjustment frequencies
- Choose parameters for the Taylor rule
- Set autocorrelation coefficients
- Unit intertemporal elasticity of substitution

```
// Calibration  
betta    = 0.995;  
eta      = 1;  
theta    = 0.78;  
phi_pi   = 1.5;  
phi_y    = .5/4;  
rho_a    = 0.8;  
rho_c    = 0.9;  
rho_i    = 0.5;  
sig      = 1;
```

## How to interact with Dynare: 2) Model definition

- Dynare allows writing the model in a very natural way
- model-block contains model declaration  
→ linear states that model is already linearised
- (+1) indicates one-period ahead variables  $X_{t+1}$  (forward-looking)
- (-1) indicates lagged variables  $X_{t-1}$  (backward-looking)
- $X_t$  is written  $X$  (no time index needed for the current period)

```
model(linear);  
  [name='Phillips Curve']  
  pie=beta*pie(+1)+kappa*y+zeps_a;  
  [name='DIS curve']  
  y=y(+1)-1/sig*(inom-pie(+1))  
  +zeps_c;  
  [name='Nominal interest rate Taylor rule']  
  inom=phi_pi*pie+phi_y*y+zeps_i;  
  zeps_a=rho_a*zeps_a(-1)+u_a;  
  zeps_c=rho_c*zeps_c(-1)+u_c;  
  zeps_i=rho_i*zeps_i(-1)+u_i;  
  
end;
```

## Timing conventions

- Time indices refer to period when variable is determined, i.e. **stock at the end of period notation**
- Capital stock used in production at time  $t$  is determined by investment at time  $t - 1$   
→ predetermined

$$K_t = (1 - \delta)K_{t-1} + I_t \quad (5)$$

$$Y_t = K_{t-1}^\alpha L_t^{1-\alpha} \quad (6)$$

- To use **stock at the beginning of period notation**

$$K_{t+1} = (1 - \delta)K_t + I_t \quad (7)$$

$$Y_t = K_t^\alpha L_t^{1-\alpha} \quad (8)$$

you can declare variable as predetermined using `predetermined_variables` declaration

## Timing conventions: Code

Example using the default convention

```
var y, k, i;  
...  
...  
model;  
y = k(-1)^alpha;  
k = i + (1-delta)*k(-1);  
...  
end;
```

Using `predetermined_variables`:

```
var y, k, i;  
predetermined_variables k;  
...  
model;  
y = k^alpha;  
k(+1) = i + (1-delta)*k;  
...  
end;
```

## Timing conventions: conditional expectations

- Conditional expectations operator  $\mathbb{E}_t$  typically not needed in Dynare as it starts from equilibrium system

$$0 = \mathbb{E}_t (f(y_{t+1}, y_t, y_{t-1}, \varepsilon_t)) \quad (9)$$

and therefore assumes conditional expectations operator in front of every equation

- $1/c = \beta/c_{t+1} * (R_{t+1})$ ;  
corresponds to

$$\frac{1}{c_t} = \mathbb{E}_t \left( \frac{1}{c_t} \right) = \mathbb{E}_t \left( \beta \frac{R_{t+1}}{c_{t+1}} \right) = \beta \mathbb{E}_t \left( \frac{R_{t+1}}{c_{t+1}} \right) \quad (10)$$

- Due to certainty equivalence at first order, we do not have to bother with this, but intricate issues can arise at higher order (e.g. Epstein-Weil-Zin preferences)
- `EXPECTATION(INTEGER) (MODEL_EXPRESSION)` allows specifying information set of expectations explicitly

# The steady state

- If not told differently, Dynare tries whether 0 is the steady state, and if not, starts a numerical solver to find the steady state
- As 0 is often infeasible for nonlinear models, always provide explicit steady state information
- Three ways to do this:
  1. compute steady state analytically and provide it to Dynare using `steady_state_model`-block
  2. provide initial guesses to start the numerical optimizer using the `initval`-block (the better, the higher the chances of finding the steady state; economic intuition provides good guidance, e.g.  $I < C < Y < K$ , with  $I = \delta K$ .)
  3. use explicit steady state file, which offers a lot of flexibility
- Note: linearized models typically do not spare you from computing the steady state, they just shift the problem to computing parameters

## How to interact with Dynare: 3) Steady state

- Our model is expressed in deviation from the steady state (around zero inflation)
- The deterministic steady state implies no shocks:  $u = 0$
- Thus, all variables are zero in the steady state

```
steady_state_model;  
pie=0;  
y=0;  
inom=0;  
zeps_a=0;  
zeps_c=0;  
zeps_i=0;  
end;
```

or

```
steady_state_model;  
end;
```

## Numerical steady state

- Dynare can solve for the steady state using initial starting values provided

```
initval;  
y=0;  
end;
```

- Inside `steady_state_model;` and `initval;`-blocks, you can define variables recursively and use auxiliary definitions
- Always use a fully analytical steady state if possible (especially for estimation)
- The steady state values stored in `oo_.steady_state` follow the declaration order
- You can access steady state values within the `model;`-block using the `STEADY_STATE (MODEL_EXPRESSION)` operator

## Steady state .m-file

- You can also calculate the steady state using your own numerical routines.
- Create a Matlab file with the same name as your model followed by `_steadystate.m`  
→ `DSGE_steadystate.m` for model file `DSGE.mod`
- The file allows a lot of flexibility like calling numerical solvers, but syntax is more involved
- Full example available [here \[LINK\]](#)

## Digression: Log-linearization vs. linearization

- Dynare linearizes the model around the steady state
- However, we often want log-linearization to express everything in percentage deviations from steady state (or the balanced growth path)
- To obtain a log-linear solution, you can
  1. use the `loglinear` option
  2. write down the model in terms of logarithmic transformations of the original variables (`exp()`-substitution)
  3. append the logarithm of model variables as separate model variables (e.g. `log_y=log(y)`)  
→ Dynare 6.x: use `var(log) y;` to append `LOG_Y`
- The last option tends to be the most preferable

## Structure of typical Dynare *.mod* file: shocks

- shocks;-block defining the covariance matrix of exogenous shocks

```
shocks;  
    var u_c  = 0.005^2;  
    var u_a  = 0.005^2;  
    var u_i  = 0.005^2;  
    //corr u_c,u_i=0.5;  
end;
```

- Also allows specifying correlations (or covariances)
- Different syntax for perfect foresight and OccBin

## Parameter dependence

- Sometimes, parameters like the slope of the Phillips Curve are functions of other parameters:

$$\kappa = \frac{(\sigma + \eta)(1 - \theta)(1 - \beta\theta)}{\theta} \quad (11)$$

- There are two ways to handle such objects:
  - As a model-local variable in the model block, using the `#` operator  
→ declaring `model_local_variable` allows setting  $\text{\LaTeX}$  name
  - setting them in a `steady_state_model`-block or file

```
model_local_variable kappa $\kappa$;  
model(linear);  
#kappa = (sig+eta)*(1-theta)*(1-betta*theta)/theta;  
  
pie=betta*pie(+1)+kappa*y+zeps_a;
```

- Accounting for these dependencies is important, especially for estimation

## Requesting computations

- Display residuals with `resid;` command
- Compute and display steady state with `steady;` command
- `check;` command to display local stability conditions

```
resid; %display residuals  
steady; %display steady state (and compute it if necessary)  
check; %display local stability information
```

- `stoch_simul;` command to
  - linearize the model around the steady state
  - compute recursive equilibrium laws of motion of the linearized model
  - analyze the model via impulse responses, stochastic simulations, moments, etc.

```
stoch_simul(order=1,irf=20,irf_plot_threshold=0) y pie inom;
```

- Commands to generate  $\text{\LaTeX}$  output

```
write_latex_dynamic_model(write_equation_tags);  
write_latex_parameter_table;  
write_latex_definitions;  
collect_latex_files;
```

- Particularly useful for debugging

# Outline

1. Intro
2. Dynare Overview
3. Modeling language
4. Invocation and results

# Running Dynare

- You need to add Dynare to your Matlab path, e.g.

```
addpath('c:/dynare/6.4/matlab')
```

- You can run a .mod-file located in MATLAB's *Current Folder* using

```
dynare FILENAME[.mod]
```

## Accessing Dynare output

- Dynare stores results, model information, and selected options in MATLAB structures (use the tab key at the MATLAB command line to access the different fields)
  - `oo_` stores results from computations (simulation results, ...)
  - `M_` stores model information (e.g. info on parameters and their values)
  - `options_` stores the selected options
  - `estim_params_` stores information on estimated parameters
  - `bayestopt_` stores information on the prior
- All output fields should be documented in the manual

# Bibliography I

- Adjemian, Stéphane et al. (2024). “Dynare: reference manual, version 6”. [Dynare Working Papers 80](#). CEPREMAP.
- Pfeifer, Johannes (2013a). “A guide to specifying observation equations for the estimation of DSGE models”. [Mimeo](#).  
<https://drive.google.com/file/d/1r890U50E3CBa6t0lj6l3hNVWEaRH5Anv/view>.  
University of the Bundeswehr Munich.
- (2013b). “An introduction to graphs in Dynare”. [Mimeo](#).  
<https://drive.google.com/file/d/1D010K5AKLWfcJX49ZPCGo58Ni2LdERw4/view>.  
University of the Bundeswehr Munich.